Royale Cars — Plano de unificação das três plataformas

Oct 13, 2026 · Grupo Royale Cars: Caio (desktop), Gabriel (web), Evaldo (mobile)

Visão geral

Hoje o Royale Cars é um sistema contado de três jeitos. A web (PHP) tem as 32 funcionalidades e as regras mais completas, mas grava num MySQL que só existe no computador de quem roda. O desktop (Electron) e o celular (Flutter) já usam o mesmo projeto Supabase, mas gravam com máscara, têm login próprio e faltam 19 e 20 funcionalidades, respectivamente.

A proposta deste documento: **um banco só no Supabase, onde ficam as regras** (validação, permissões, cálculo do aluguel e histórico), e **três aplicativos que cuidam só das telas**. A web serve de modelo para as regras; o desktop e o celular entram com o que só eles têm (grupos de permissão, quilometragem, assinatura, contrato em PDF e checklist).

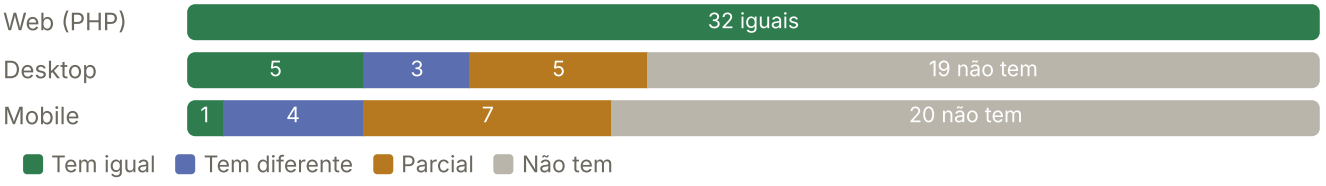
O trabalho foi dividido em **7 fases e cerca de 50 pacotes**. Cada pacote é uma mudança pequena, testada e com mensagem de commit pronta, para o histórico do GitHub mostrar o projeto evoluindo passo a passo.

Como usar este documento:

1. Leiam **Onde cada plataforma está** e **O que eu conferi nos códigos**: é o diagnóstico.
2. As seções **Arquitetura**, **O que levar de cada plataforma** e **Padrão comum** são a proposta.
3. **Ordem do trabalho** e **Pacotes** são o planejamento: o que vem antes, quem faz o commit e quando o banco precisa de vocês.
4. **Como funciona cada pacote** e **Git e GitHub** respondem a pergunta dos commits.
5. Antes de eu mexer em qualquer coisa, respondam as **Decisões pendentes** e a lista **O que eu preciso de vocês agora** (última parte do documento).

Onde cada plataforma está

Contagem do checklist de 32 funcionalidades da web, tirada dos três documentos de vocês:



Assunto	Web (PHP)	Desktop (Electron)	Mobile (Flutter)
Banco	MySQL local (XAMPP), 8 tabelas	Supabase, 5 tabelas	Mesmo Supabase do desktop + 1 tabela e colunas próprias

Assunto	Web (PHP)	Desktop (Electron)	Mobile (Flutter)
Login	Tabela própria, bcrypt no PHP, sessão no servidor	Tabela `usuarios`, bcrypt dentro do app	Igual ao desktop; não confere conta desativada
Permissões	3 cargos, 13 chaves por pessoa	2 cargos, 14 chaves por grupo	Nenhuma: todo mundo vê tudo
Formato dos dados	Sem máscara (só números)	Com máscara	Com máscara
Status	`Em manutenção`, `Em andamento` / `Finalizada` / `Cancelada`	`Em Manutenção`, `Ativo` / `Concluído`	Igual ao desktop
Onde ficam as regras	PHP + MySQL	No app, poucas	No app, médias
Só ela tem	Manutenção, alertas, relatórios, histórico, painel do chefe	Grupos de permissão, bandeja do sistema, multiempresa	Quilometragem, assinatura, contrato PDF, checklist, taxas

Em resumo: a web está mais completa, mas é a que está mais longe do banco. O desktop e o celular já estão no banco, mas estão longe das regras.

O que eu conferi nos códigos

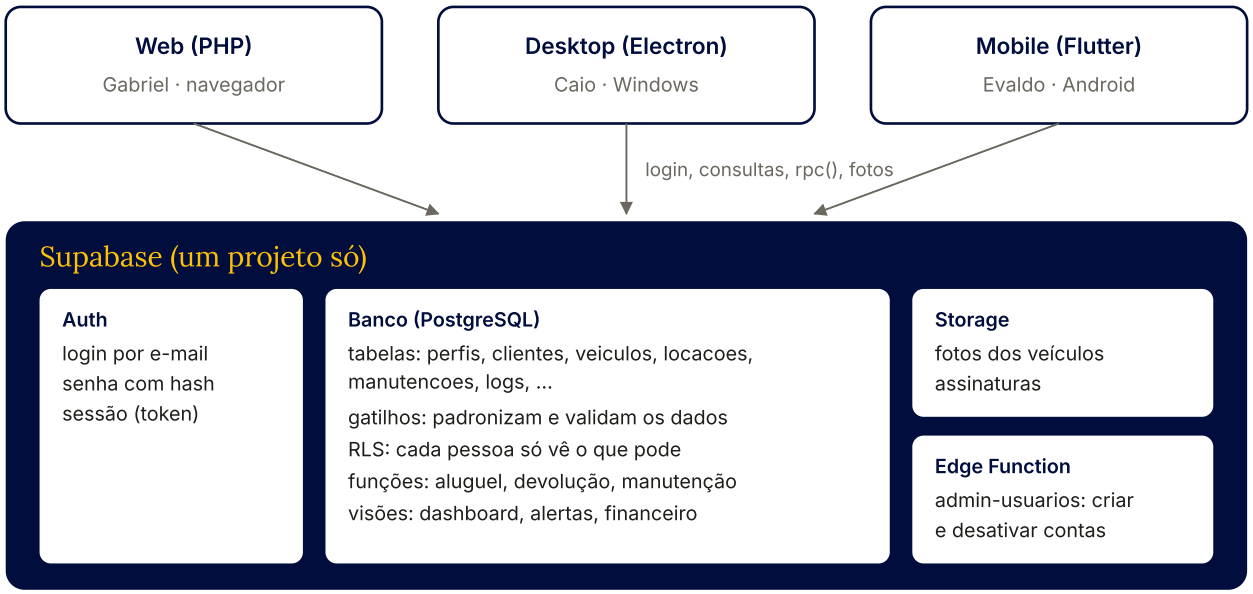
Abri os três zips para confirmar os PDFs. Eles batem com o código. Encontrei seis pontos que mudam o plano:

O que encontrei	Por que importa	O que fazemos
O script do banco unificado não está no zip. O documento da web cita `supabase/royale_cars_supabase.sql`, mas o zip só tem o `royale_cars.sql` do MySQL.	Toda a unificação depende desse script. Sem ele não há onde os três apps se conectarem.	Se vocês tiverem esse arquivo, me mandem. Se não, eu escrevo na Fase 1, em partes.
O zip da web tem duas versões. `royale-cars-web` (1º de setembro, antiga) e `Royale-Cars-TCC/royale-cars-web` (9 de outubro, a completa).	Se alguém copiar a pasta errada, perde metade do sistema.	Usar só a de 9 de outubro. A antiga não entra no repositório.
O zip do mobile tem 169 MB, e 157 MB são a pasta `build/`.	Essa pasta é gerada sozinha pelo Flutter. Se for para o GitHub, o repositório fica pesado e o push pode falhar.	Colocar `build/` no `.gitignore` (pacote 0.1).
Nenhum zip tem pasta `.git`.	Não sei se vocês já têm repositório, nem se é um ou três.	Pergunta na lista do final.

O que encontrei	Por que importa	O que fazemos
A chave do Supabase está escrita no código do desktop e do celular, e o celular lê a linha inteira de `usuarios`, inclusive o hash da senha.	Essa chave (`publishable`) pode ser pública, *desde que o RLS esteja ligado*. Se estiver desligado, qualquer pessoa com o app lê e altera tudo.	A Fase 1 liga o RLS em todas as tabelas. Até lá, conferir no painel (consulta na lista do final).
O que eu consigo testar aqui: tenho PHP 8.3, PostgreSQL 16 e Node 22. Não tenho Flutter.	Consigo rodar o script do banco num Postgres de teste e conferir PHP e JavaScript antes de entregar. O Dart eu não consigo compilar.	Nos pacotes do mobile, o Evaldo roda `flutter run` e me manda os erros, se houver.

Arquitetura proposta: regras no banco, telas nos apps

Pensem no banco como a **portaria de um prédio**. Hoje cada app tem o seu porteiro: o da web confere CPF, o do desktop não confere, o do celular confere pela metade. Quem entra pelo porteiro mais fraco grava o dado errado, e os outros dois veem o erro. A solução é ter **um porteiro só, dentro do banco**. Os apps continuam com as suas telas e máscaras (para ajudar quem digita), mas quem decide se o dado entra é o banco.



Os três apps falam com o mesmo Supabase. Tudo que é regra fica na caixa azul, escrito uma vez só, em SQL.

Por que trocar o login próprio pelo Supabase Auth

É a maior mudança para o desktop e o celular, então vale explicar. Com o login próprio, o app usa a mesma chave pública para todo mundo; o banco não sabe *quem* está pedindo os dados, então não consegue aplicar permissão nenhuma. Com o Supabase Auth, cada pessoa recebe um "crachá" (token) ao entrar, e o banco passa a saber quem é quem.

Ponto	Manter login próprio (tabela `usuarios`)	Supabase Auth (recomendado)
Quem protege os dados	Só a tela do app. Pela chave pública dá para ler tudo, inclusive os hashes de senha.	O banco (RLS). Desativar alguém corta o acesso na hora, nas três plataformas.
Trabalho em cada app	Nenhum agora, mas cada app continua conferindo senha, bloqueio e permissão sozinho (três vezes).	Uma chamada (`signInWithPassword`) nos SDKs de JavaScript e Dart; na web, uma requisição HTTP.
Senhas atuais	Continuam.	O Supabase aceita importar hash bcrypt, então ninguém precisa criar senha nova (se os dados forem mantidos).
Na banca	Difícil defender a segurança.	Dá para mostrar: "desativei o Lucas na web e o celular dele saiu".

O que muda para cada app, em uma frase

- **Desktop:** continua com o `supabase-js`; troca o login, passa a chamar as funções do banco (`supabase.rpc`) para aluguel e manutenção, e lê as visões no dashboard.
- **Web:** sai do MySQL. O PHP passa a falar com o Supabase pela API (um arquivo `includes/supabase.php` com funções como `sb_select` e `sb_rpc`). As telas ficam quase iguais; o que muda é a parte que busca e grava os dados.
- **Mobile:** continua com o `supabase_flutter`; troca o login, tira o `id_empresa` das consultas e passa a usar as mesmas funções e visões.

O que levar de cada plataforma

Nenhuma das três está "certa" sozinha. A web é a base das regras, mas o desktop e o celular têm ideias que a banca vai gostar de ver nas três.

Da web

a base de tudo

- Formatos gravados no banco (CPF, telefone, placa sem máscara)
- Validações: CPF e CNH com dígito, DDD, placa, chassi, nome
- Status do veículo só por ações
- Cálculo do aluguel com diária gravada, data real e atraso
- Cancelamento com motivo
- Módulos: Manutenção, Alertas, Financeiro, Relatórios + CSV
- Segurança: bloqueio por tentativas, senha temporária, saída por inatividade
- Painel do chefe: histórico de ações e tentativas de login

Do desktop

organização da equipe

- Grupos de permissão como **modelos** (Gerente, Vendedor, Financeiro) que preenchem as caixas da pessoa
- Tela "Ver colaboradores" do grupo e mover de grupo
- Endereço do cliente (útil no contrato)
- Modal de confirmação padrão (vermelho = perigo, verde = confirmar)
- Botão "Arquivados" que alterna a lista
- Listas que atualizam sozinhas
- Bandeja do sistema (só desktop)

Do mobile

o atendimento no balcão

- **Quilometragem** no veículo e na locação (pedido da banca)
- **Assinatura digital** do cliente (pedido da banca)
- **Contrato em PDF** com a assinatura, que vira o comprovante das três
- **Checklist de inspeção** (motor, freios, pneus, óleo) dentro da manutenção
- Taxas extras na devolução, com motivo
- Relatório de desempenho (taxa de ocupação)
- Sobre Nós (apresenta o grupo)

O que sai das três

Sai	Onde existe hoje	Por quê
"Criar conta" público	Desktop e mobile	Qualquer pessoa criava uma locadora. Só o chefe cadastra pessoas.
"Esqueci minha senha" só com o e-mail	Desktop	Quem soubesse o e-mail de alguém trocava a senha dele. O chefe redefine com senha temporária.
Multiempresa (`id_empresa`)	Desktop e mobile	Se o grupo escolher empresa única (decisão 1), simplifica o banco e todas as consultas.
Status do veículo no formulário	Desktop e mobile	Era o que deixava carro "preso" em manutenção com aluguel aberto.
Login por nome de usuário	Desktop e mobile	O Supabase Auth entra por e-mail.
`bcrypt` dentro dos apps	Desktop e mobile	A senha passa a ser conferida pelo Supabase Auth.
Formulário de Contato que finge enviar	Mobile	Na banca, um botão que não faz nada pega mal.

Padrão comum

As regras detalhadas (algoritmos de CPF e CNH, limites de datas, cálculo) continuam valendo como estão no documento da web. Aqui fica só o resumo do que as três precisam escrever igual.

Assunto	Padrão
Formato no banco	CPF e CNH com 11 números; telefone com DDD, só números; e-mail em minúsculas; placa em maiúsculas sem hífen; chassi com 17 caracteres; dinheiro com 2 casas; datas ano-mês-dia. A máscara é só na tela.
Status do veículo	`Disponível` · `Alugado` · `Em manutenção` (m minúsculo). Muda só por ações.
Status da locação	`Em andamento` · `Finalizada` · `Cancelada`. "Atrasada" é calculado, não gravado.
Status da manutenção	`Em andamento` · `Concluída`
Cargos	`chefe` · `gerente` · `funcionario`
Permissões	As 13 chaves da web, por pessoa. Os grupos do desktop viram modelos que marcam as caixas.
Senha	Mínimo de 8 caracteres, com letra e número.
Cores	Azul `#010d3d` e dourado `#FFC107` (as três já usam).

Nomes de tela (proposta)

Hoje a mesma tela tem nomes diferentes. Proposta: usar o nome que já aparece em duas das três plataformas, ou o que bate com o nome da tabela.

Web hoje	Desktop hoje	Mobile hoje	Proposta para as três
Aluguéis	Locações	Nova Locação / Devoluções	**Locações** (tabela `locacoes`)
Veículos	Frota	Frota / Cadastro Veículos	**Frota**
Financeiro	Financeiro	Finanças	**Financeiro**
Painel Administrativo	Administração	(não tem)	**Administração**
Minha conta	Configurações	(não tem)	**Minha conta**

Banco unificado: as tabelas

É o banco da proposta da web com o que o desktop e o celular trazem. As colunas novas em relação à web estão em negrito.

Tabela	O que guarda	Veio de
`perfis`	Nome, e-mail, cargo, ativo, troca de senha obrigatória, último acesso (o login fica no Supabase Auth)	Web

Tabela	O que guarda	Veio de
`usuario_permissoes`	As 13 permissões de cada pessoa	Web
`modelos_permissao`	Grupos-modelo: nome, descrição e lista de chaves	Desktop (grupos)
`clientes`	Nome, CPF, CNH, validade da CNH, e-mail, telefone, **endereço**, arquivado	Web + desktop
`veiculos`	Marca, modelo, ano, placa, chassi, diária, foto, status, **quilometragem**, arquivado	Web + mobile
`locacoes`	Cliente, veículo, datas, diária da época, valores, atraso, quem registrou e encerrou, **km de saída e retorno, assinatura, taxas extras e motivo**	Web + mobile
`manutencoes`	Defeito, oficina, datas, custo, **checklist (motor, freios, pneus, óleo)**	Web + mobile (laudo)
`logs`, `login_tentativas`	Histórico de ações e tentativas de login	Web

Ordem do trabalho

Vocês pensaram em deixar uma plataforma pronta e depois passar para as outras. Proponho um ajuste: **primeiro conectar as três ao banco novo, depois completar as funcionalidades**. O motivo é o teste. Se o desktop ficar 100% pronto antes de a web e o celular entrarem, um erro no banco só aparece semanas depois, quando já tem muito código em cima dele. Conectando as três cedo, o teste cruzado ("alugar no celular, devolver no desktop, conferir na web") acontece no meio do projeto, quando ainda é barato corrigir.



"Conectar" = a plataforma passa a usar o banco novo com as telas que já tem. "Completar" = construir o que falta. Os marcos (M) são os momentos de teste juntos.

Por que o desktop primeiro

- **Já fala com o Supabase pelo `supabase-js`.** O salto é o menor dos três, então o banco novo é testado mais cedo.
- **Ele tem a tela de colaboradores**, então já testa logo a parte mais delicada: o chefe criando e desativando contas pela Edge Function.
- **O Caio está acompanhando comigo** e consegue testar cada pacote na hora.
- **O fluxo de login escrito em JavaScript serve de modelo** para o Dart e o PHP, que vêm depois.

A web vem em segundo porque é a referência das regras: assim que ela entra, dá para comparar desktop e web lado a lado no mesmo banco. O celular vem por último porque é o que tem mais coisa faltando e porque eu não consigo compilar Flutter aqui; quando ele chegar, o banco já terá sido testado pelos outros dois.

Tempo estimado

Ainda não sei a data da entrega, então a tabela usa semanas relativas. É estimativa: depende de quanto tempo vocês levam para testar cada pacote.

Fase	Pacotes	Estimativa	Depende de vocês para
0 · Preparo	2	1 a 2 dias	Responder as decisões, criar ou indicar o repositório
1 · Banco	10	1 semana	Rodar os scripts no Supabase e publicar a função
2 · Desktop conectar	8	1 semana	Testar cada pacote no Windows
3 · Web conectar	8	1 semana	Testar no XAMPP
4 · Mobile conectar	8	1 semana	Compilar e testar no celular
5 · Completar e alinhar	~10	1 a 2 semanas	Testar e validar o visual
6 · Fechar	3	2 a 3 dias	Ensaiar a apresentação

Total aproximado: 5 a 7 semanas. Se a entrega for antes disso, a Fase 5 é a que encolhe: o celular pode ficar focado no atendimento (decisão 9) e alguns relatórios ficam só na web.

Pacotes (cada pacote = um commit)

Legenda: `vocês` = precisa de alguém rodar algo no Supabase ou no painel; quem commita é uma sugestão para o histórico refletir os papéis do grupo.

Fase 0 · Preparo

#	Pacote	Mensagem de commit	Quem commita
0.1	Estrutura do repositório, `.gitignore` (build, node_modules, segredos), README geral e os PDFs em `docs/`	`chore: organiza repositório unificado do Royale Cars`	Caio
0.2	Permissão de internet no APK final do Android	`fix(mobile): libera internet no APK de release`	Evaldo

Fase 1 · Banco unificado

Cada arquivo SQL é um pacote, numerado na ordem em que roda. Eu testo todos juntos num PostgreSQL 16 aqui antes de entregar, simulando o login do Supabase.

#	Pacote	Mensagem de commit	Quem commita
1.1	`banco/01_estrutura.sql`: tabelas, chaves e restrições	`feat(banco): cria tabelas do banco unificado`	Caio
1.2	`02_validacoes.sql`: CPF, CNH, telefone, placa, chassi e padronização automática	`feat(banco): valida e padroniza documentos e placas`	Gabriel
1.3	`03_permissoes.sql`: `tem_permissao`, RLS em todas as tabelas, modelos de grupo	`feat(banco): permissões por pessoa e RLS`	Caio
1.4	`04_login.sql`: `iniciar_sessao`, bloqueio, tentativas, troca de senha, `criar_chefe`	`feat(banco): sessão, bloqueio e tentativas de login`	Evaldo
1.5	`05_locacoes.sql`: registrar aluguel, devolução e cancelamento (com km, assinatura e taxas)	`feat(banco): funções de retirada, devolução e cancelamento`	Gabriel
1.6	`06_manutencoes.sql`: abrir (com checklist) e concluir manutenção	`feat(banco): funções de manutenção com checklist`	Evaldo
1.7	`07_historico_visoes.sql`: logs automáticos e visões de locações, frota, faturamento e alertas	`feat(banco): histórico automático e visões dos painéis`	Caio
1.8	`08_storage.sql`: buckets `veiculos` (público) e `contratos` (privado)	`feat(banco): armazenamento de fotos e assinaturas`	Gabriel
1.9	Edge Function `admin-usuarios` (criar, redefinir senha, ativar, desativar, alterar e-mail)	`feat(supabase): função de administração de usuários`	Caio
1.10	`09_dados_demo.sql` e roteiro de testes do banco (ou migração dos dados atuais, se decidirem manter)	`test(banco): dados de demonstração e roteiro de testes`	Evaldo

M1 · banco no ar `vocês` Backup do banco atual, rodar os scripts no SQL Editor, desligar o cadastro público, criar o chefe e publicar a função. Eu mando o passo a passo com cada clique.

Fase 2 · Desktop conectado (commits do Caio)

#	Pacote	Mensagem de commit
2.1	Login pelo Supabase Auth, bloqueio e registro de tentativas; sai "Criar conta", "Esqueci minha senha" e `bcryptjs`	`feat(desktop): login pelo Supabase Auth`
2.2	Troca obrigatória de senha, saída por inatividade e Minha conta salvando de verdade	`feat(desktop): troca de senha obrigatória e saída automática`
2.3	Menu e processo principal pelas 13 chaves; grupos viram modelos	`refactor(desktop): permissões no padrão das 3 plataformas`
2.4	Colaboradores pela Edge Function (senha temporária, ativar, desativar)	`feat(desktop): administração de colaboradores pelo Supabase`
2.5	Clientes sem máscara no banco, validade da CNH, erros do banco na tela, e-mail e endereço salvando na edição	`refactor(desktop): clientes no padrão unificado`
2.6	Frota: sem status no formulário, placa sem hífen, ano, chassi e km	`refactor(desktop): frota no padrão unificado`
2.7	Locações pelas funções: retirada com km; devolução com data real, atraso, km, taxas e defeito; cancelar	`feat(desktop): locações pelas funções do banco`
2.8	Dashboard e Financeiro lendo as visões (corrige o ticket médio)	`refactor(desktop): painéis lendo as visões do banco`

M2 · primeiro app no banco novo. Caio faz o roteiro de testes no desktop; o que der errado no banco é corrigido antes de a web começar.

Fase 3 · Web conectada (commits do Gabriel)

#	Pacote	Mensagem de commit
3.1	`includes/supabase.php` (login, consultas, rpc, upload) e configuração fora do Git	`feat(web): cliente do Supabase em PHP`
3.2	Login, sessão, bloqueio e troca de senha pelo Supabase Auth	`feat(web): login pelo Supabase Auth`
3.3	Clientes, ficha e arquivados (+ endereço)	`refactor(web): clientes no Supabase`
3.4	Frota, foto no Storage e quilometragem	`refactor(web): frota e fotos no Supabase`

#	Pacote	Mensagem de commit
3.5	Locações e manutenção pelas funções (+ km, taxas, checklist, assinatura no comprovante)	`feat(web): locações e manutenção pelas funções do banco`
3.6	Dashboard, alertas, financeiro e relatórios pelas visões	`refactor(web): painéis e relatórios pelas visões`
3.7	Administração pela Edge Function; histórico e tentativas	`feat(web): administração pelo Supabase`
3.8	Remove a conexão MySQL; `royale_cars.sql` vai para `docs/legado/`	`chore(web): remove MySQL`

M3 · teste cruzado desktop ↔ web. Cadastrar cliente na web e alugar no desktop; desativar alguém na web e ver o desktop recusar.

Fase 4 · Mobile conectado (commits do Evaldo)

#	Pacote	Mensagem de commit
4.1	Login pelo Supabase Auth, sessão salva no aparelho, troca obrigatória; sai "Criar conta" e `bcrypt`	`feat(mobile): login pelo Supabase Auth`
4.2	Permissões nos atalhos e botões; tira o `id_empresa` das consultas	`feat(mobile): menu por permissão`
4.3	Clientes no padrão + tela de arquivados	`refactor(mobile): clientes no padrão unificado`
4.4	Frota no padrão + arquivados + foto pela câmera	`refactor(mobile): frota no padrão unificado`
4.5	Nova locação pela função; assinatura vai para o Storage	`feat(mobile): retirada pela função com assinatura no Storage`
4.6	Lista de locações com abas, devolução pela função, cancelar e contrato PDF a qualquer momento	`feat(mobile): devolução, cancelamento e contrato`
4.7	Manutenção: checklist + oficina + botão Liberar com custo	`feat(mobile): manutenção completa`
4.8	Painel com cartões e alertas, Financeiro pelas visões e Minha conta	`feat(mobile): painel, alertas e minha conta`

M4 · teste cruzado das três. Alugar no celular (com assinatura), devolver no desktop, conferir o comprovante e o financeiro na web.

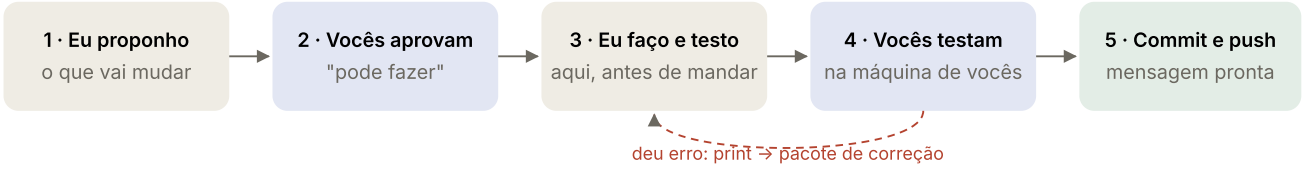
Fase 5 · Completar e alinhar

#	Pacote	Quem commita
5.1	Desktop: tela de Manutenção	Caio
5.2	Desktop: Alertas e dashboard completo (gráfico e barra da frota)	Caio
5.3	Desktop: fichas de cliente e veículo, foto do veículo	Caio
5.4	Desktop: contrato/comprovante com assinatura para imprimir	Caio
5.5	Desktop: Relatórios com filtros, impressão e CSV	Caio
5.6	Desktop: histórico de ações e tentativas de login	Caio
5.7	Mobile: relatórios só de leitura com filtros; Sobre Nós; tira o Contato falso	Evaldo
5.8–5.10	Nomes de tela, textos e ícones iguais (um commit por plataforma)	Cada um na sua

Fase 6 · Fechamento

#	Pacote	Quem commita
6.1	Dados de demonstração finais e roteiro de apresentação nas três plataformas	Gabriel
6.2	README e documentação final em `docs/`	Evaldo
6.3	Versão 1.0 marcada no GitHub (tag e release)	Caio

Como funciona cada pacote



A aprovação: por pacote ou por fase

Vocês pediram para eu sempre consultar antes. Para não travar o ritmo, sugiro aprovar **por fase**: no começo de cada fase eu mostro a lista de pacotes e o que cada um muda; com o "pode fazer", eu entrego os pacotes em sequência e só paro para perguntar quando aparecer algo que não estava

previsto (uma regra nova, algo que mude o banco, uma dúvida de tela). Se preferirem aprovar um por um, também funciona; só fica mais lento.

O que vem em cada entrega

Parte	O que é
O zip	A pasta da plataforma inteira, já com a mudança (sem `node_modules` e sem `build`).
O que foi feito	Lista curta dos arquivos que mudaram e o que mudou em cada um.
Como testar	Passo a passo do teste ("entre como Lucas, tente abrir Financeiro, deve aparecer Acesso negado").
Mensagem de commit	Pronta para copiar.
Precisa de vocês?	Se tiver algo para rodar no Supabase, vem aqui, com os cliques.
Aula do pacote	Explicação do raciocínio do código, do jeito das nossas aulas: o problema, a ideia, e como ela vira linha de código. É o que vocês vão precisar para responder a banca.

Como aplicar um pacote (sem perder arquivo apagado)

Se vocês só extraírem o zip por cima da pasta, os arquivos que eu apaguei continuam lá. Para o Git enxergar tudo, o jeito seguro é:

1. Na pasta da plataforma, apagar tudo **menos** `.git`, `node_modules` (desktop) e `build` (mobile).
2. Extrair o zip do pacote dentro dela.
3. Rodar `git status`: aparecem os arquivos alterados, novos e apagados. Confiram se bate com o "O que foi feito".
4. Testar.
5. Fazer o commit:

```
git add -A
git commit -m "feat(desktop): login pelo Supabase Auth"
git push
```

> **Um jeito mais rápido**
> Esta conversa está ligada ao computador do Caio pelo app do Claude. Se vocês adicionarem a pasta do repositório (botão "Adicionar pasta" no app), eu escrevo os arquivos direto nela, sem zip. Vocês só testam, rodam `git status` e fazem o commit. Continua sendo um pacote por vez, e eu continuo pedindo aprovação antes.

Git e GitHub

Um repositório só (recomendado)

Como agora é um sistema só, um repositório com as três plataformas e o banco mostra isso para os professores e deixa o histórico contar a história inteira em ordem. Se vocês já têm três repositórios com histórico, dá para manter os três e criar um quarto para o banco; me digam qual é o caso.

```
royale-cars/
├── banco/                 scripts SQL numerados (01_estrutura.sql, 02_...)
├── supabase/functions/    admin-usuarios (Edge Function)
├── desktop/               Electron
├── web/                   PHP
├── mobile/                Flutter
├── docs/                  os PDFs do projeto e este plano
├── .gitignore             build/, node_modules/, config com senha
└── README.md              o que é o projeto e como rodar cada parte
```

Padrão das mensagens de commit

Formato `tipo(área): o que mudou`, em português e no imperativo, como nos pacotes acima. Os tipos:

Tipo	Quando usar	Exemplo
`feat`	Funcionalidade nova	`feat(mobile): manutenção completa`
`fix`	Correção de erro	`fix(desktop): salva e-mail ao editar cliente`
`refactor`	Muda o código sem mudar o que a pessoa vê	`refactor(web): clientes no Supabase`
`chore`	Organização, configuração	`chore: organiza repositório unificado`
`docs` / `test`	Documentação / testes	`docs: roteiro de apresentação`

Branches e pull requests (opcional, mas conta ponto)

Uma branch por fase (`fase-2-desktop`, `fase-3-web`…), com os commits dos pacotes, e no fim da fase um **pull request** para a `main` aprovado por outro membro do grupo. Isso mostra para os professores que um revisou o trabalho do outro. Se o Git ainda é novidade para alguém, dá para começar só na `main` e adotar as branches na Fase 2.

Cuidados

- **Nunca commitar a chave `service_role`** do Supabase nem senhas. A chave `publishable` pode ir; a de serviço só existe no painel do Supabase.
- A configuração da web com dados de conexão fica num arquivo fora do Git (`config.local.php`), com um modelo `config.exemplo.php` no repositório.

- Cada pessoa commita do próprio computador, com o próprio usuário do GitHub (`git config user.name` e `user.email`), para os commits aparecerem no nome certo.
- Confiram as regras do curso sobre uso de IA. Se for preciso declarar, uma linha no README ("desenvolvido com apoio do Claude, da Anthropic") resolve.
- Quem commita precisa entender o pacote: a banca vai perguntar sobre o código. Por isso cada entrega vem com a aula.

Riscos e como evitamos

Risco	Como evitamos
Perder os dados que já estão no Supabase	Backup em CSV antes de qualquer script; o script só apaga e recria se vocês disserem que os dados são de teste (decisão 3).
Erro no banco aparecer tarde	Script testado aqui num PostgreSQL 16, roteiro de testes no pacote 1.10 e teste cruzado a cada plataforma que entra.
Código Flutter que eu não consigo compilar	Pacotes pequenos; o Evaldo roda `flutter run` e manda o erro; eu corrijo num pacote `fix`.
Edge Function ser uma ferramenta nova	Publicação pelo próprio painel do Supabase, com passo a passo. Se não der, há o plano B: uma função SQL que só o chefe chama.
Banco aberto até a Fase 1 terminar	Conferir o RLS agora (consulta abaixo). Se estiver desligado, só usar dados de teste até lá.
Prazo apertado	A Fase 5 é a que encolhe; as Fases 1 a 4 já entregam as três plataformas no mesmo banco, com as mesmas regras.

Decisões pendentes

Juntei as decisões dos três documentos e tirei as repetidas. `trava o banco` precisa de resposta antes da Fase 1; `pode esperar` dá para decidir durante. Em cada uma está a minha recomendação. Basta responder com o número e "ok" ou a escolha de vocês.

#	Decisão	Recomendação	
1	Uma empresa só ou várias (`id_empresa`)?	**Uma só.** Simplifica o banco e as consultas; o TCC não precisa de várias locadoras.	trava o banco
2	Login: Supabase Auth ou manter o próprio?	**Supabase Auth** nas três, só por e-mail (ver Arquitetura).	trava o banco
3	Os dados atuais do Supabase podem ser apagados?	Se forem só teste, **apagar e recriar** com dados de demonstração. Se não, migro (preciso das consultas).	trava o banco
4	Grupos de permissão do desktop	**Viram modelos** que marcam as caixas da pessoa. A permissão continua por pessoa.	trava o banco
5	Quilometragem nas três?	**Sim** (pedido da banca).	trava o banco

#	Decisão	Recomendação	
6	Onde fica a assinatura e quem colhe?	**Arquivo no Storage**, colhida no celular; web e desktop só mostram. Obrigatória só no celular.	trava o banco
7	Taxas/multas manuais na devolução	**Ficam**, como "taxas extras" com motivo, separadas das diárias e do atraso.	trava o banco
8	Endereço do cliente	**Nas três** (vai no contrato), opcional.	trava o banco
9	O celular terá todas as telas?	**Foco no atendimento**: clientes, frota, retirada, devolução, manutenção, alertas; financeiro e relatórios só leitura; administração só na web e no desktop.	pode esperar
10	Devolução adiantada	**Cobrar só os dias usados** (regra da web).	pode esperar
11	Quem troca o e-mail de login	**Só o chefe**, pela Administração.	pode esperar
12	Nomes de tela	A tabela da seção Padrão comum.	pode esperar
13	Sobre Nós e Contato	**Manter o Sobre Nós** e tirar o formulário de Contato.	pode esperar

O que eu preciso de vocês agora

1. **Respostas das decisões 1 a 8.**
2. **A data da entrega e da apresentação**, para eu transformar as semanas em datas.
3. **O repositório:** já existe? É um ou três? Se não existe, eu monto a estrutura no pacote 0.1 e vocês criam o repositório vazio no GitHub.
4. **O script** `royale_cars_supabase.sql`, se alguém tiver. Se não, sem problema: eu escrevo na Fase 1.
5. **Quem tem acesso ao painel do Supabase** (projeto `rhdmiqcfksekmrtgnjgo`). Essa pessoa roda as consultas abaixo e me manda o resultado (pode ser print).

Consultas para rodar no SQL Editor do Supabase (só leem, não mudam nada):

```
-- 1) Estrutura: tabelas, colunas e tipos
select table_name, column_name, data_type, is_nullable
from information_schema.columns
where table_schema = 'public'
order by table_name, ordinal_position;

-- 2) O RLS está ligado?
select c.relname as tabela, c.relrowsecurity as rls_ligado
from pg_class c join pg_namespace n on n.oid = c.relnamespace
where n.nspname = 'public' and c.relkind = 'r'
order by 1;

-- 3) Quantos registros existem (rodar só as tabelas que existirem)
select 'usuarios' as tabela, count(*) from usuarios
union all select 'clientes', count(*) from clientes
union all select 'veiculos', count(*) from veiculos
union all select 'locacoes', count(*) from locacoes
union all select 'grupos_permissao', count(*) from grupos_permissao
union all select 'laudos_manutencao', count(*) from laudos_manutencao;
```

Acompanhamento:

- [ ] Decisões 1 a 8 respondidas
- [ ] Data de entrega informada
- [ ] Repositório definido (pacote 0.1 aplicado)
- [ ] Consultas rodadas e backup feito
- [ ] M1 · banco unificado no ar
- [ ] M2 · desktop no banco novo
- [ ] M3 · teste cruzado desktop ↔ web
- [ ] M4 · teste cruzado das três
- [ ] Fase 5 concluída
- [ ] Versão 1.0 e ensaio da apresentação